Method of automatic identification of ruler scales on images to solve the problem of estimating the geometric dimensions of certain objects

Bohdan Lukashchuk
Department of Information Technologies
Ukrainian National Forestry University
Lviv, Ukraine
bohdan.lukashchuk@gmail.com

***Abstract*— Detection of pixels to real-world unit correspondence on the image is an important step for the further analysis of the geometrical characteristics of the objects on the image. In this paper, I propose an iterative method of automatic identification of the ruler scales on the image based on digit recognition. Experiments showed a mean absolute error of 6.63 px and median absolute error of 4 px on high-resolution images (3104x1746 px) compared to the manual measurement results. I focus on the medical domain images, but this method can be used in many others domains like forensic, museum business, etc.**

Keywords—digit detection, ruler scale detection, image processing, wound measurement

I. INTRODUCTION

The task of determining the size of certain objects on the images is quite common and well known in multiple areas. Different methods allow us to solve it quickly and with sufficient accuracy. However, their application usually requires knowledge of the distance to the object, as well as the parameters of the lens of a camera, at least the focal length. If this information is unavailable, this task can be solved using a reference object with a known size and distance to it. The reference object should be placed near the object of interest, on the same plane, and at the same distance from the camera. In most practical cases, approximate manual methods are used, which are primarily characterized by a large number of subjective errors. For example, in medicine, doctors place a measuring ruler next to the patient's wound. This allows us to estimate the size of the object under study. Moreover, it is important to note that currently there are different archives and various databases with a huge number of such images. The main problem here is that such an assessment is performed manually and is very subjective and inaccurate. This creates an important task – accurately and automatically estimate the size of certain objects in the images if the images contain the reference ruler. Obviously, such a task is divided into at least two subtasks. The essence of the first is that it is necessary to identify the numerical scale of the ruler, and at the same time, it is necessary to determine the price of all divisions plotted on this ruler. The second task is to determine the geometric dimensions of the object of study by comparing it with an already identified ruler. The main goal of this work is to describe a new method that solves the first task – identification of the ruler scale on the image, to determine the dots per centimeter (DPC) or dots per inch (DPI), where dots is used as a synonym to pixels.

The focus on medical imaging is due to the importance of the task of wound measurements which are used for checking the effectiveness of treatment and during performing surgical interventions.

The task of detecting DPC issues in many areas.

Abhir Bhalerao et al [1] solve the ruler detection problem in the field of forensic images. As well as in my work, they do this to reduce the amount of routine manual work. Authors detect the location of the ruler in the image and then use spectral analysis to estimate the direction and wavelength of the ruler graduations. They use a two-stage approach, where at first, ruler orientation is detected, using 2D Discrete Fourier Transformation (DFT) based on the candidate point, and after that image model is created to estimate the ruler graduation spacing. The ruler model is based on the sine curve, created using specific orientation, wavelength, and phase.

Dmitry Konovalov et al [2] report a method that solves ruler scale detection for automatic scaling of fish images that may be applicable for breeding manufacturing. They solve the problem of detecting the ruler or its part but do not extracting ruler interval distance in a fractional number of pixels per ruler's graduation. The method consists of a few stages: using a set of steps, divide an image into blocks of the same size (applying a grid with a given step); for each block apply DFT to detect blocks that contain parts of the ruler, where lots of parallel lines detect ruler; combine this candidate blocks to get the final result.

Fourier analysis is also used by Michael Herrmann et al [3] to detect the actual size of ancient coins for museum catalogues.

II. METHODS

The proposed method can be divided into three separate steps where the output of the previous step is an input to the next one:

- Digits detection.
- Detection of laying on the same line digits.
- Ruler scale detection.

A. *Digits detection*

The overall idea of this method is to perform object detection task in order to detect and identify digits that are present on the image. The hypothesis is that at least some of the detected digits are placed above or below scale marks on the ruler and can be used as a reference points for scale detection.

The main output of this step is a set of bounding boxes. Each bounding box should be represented in a specific format, with "xmin", "ymin", "xmax", "ymax" and "name" properties - Fig. 1, where the first four tokens hold corresponding min

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

and max bounding box coordinates and "name" property holds a detected digit label.

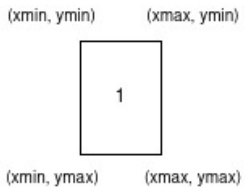


Fig. 1. Detected digit bounding box format

There is no strict requirement on the digit detection method. Methods that are based on deep convolutional neural networks can be easily trained on the custom data, so I decided to use one of them.

Currently, in deep learning, there are two major classes of object detection algorithms: one-stage and two-stage object detectors. Two-stage object detectors use separate calculations for localization – detecting possible object regions, and for classification of these regions, Faster R-CNN [4] is one of the best-known architectures of this kind. One-stage detectors do localization and classification in one go. A single neural network predicts bounding boxes and class probabilities directly from full images in one evaluation. YOLO [5] belongs to the one-stage object detectors, and it is the first well-known network of this kind. The biggest advantage of YOLO is its speed; according to the original paper of the first version of this network, it can process images in real time with 45 frames per second [5]. Despite that, there are other important advantages of the YOLO approach, like using the entire image during training and testing, but not only some regions and learning generalizable representation of objects [5]. While also having some disadvantages and limitations, YOLO passed a long path of development, and the 5-th version I use [6] has lots of them fixed and overall performance greatly improved.

It was decided to use pre-trained YOLOv5 [6] and train it on the new data.

I labeled 184 wound images with different rulers, collected from [7], and blurred all sensitive information to maintain patient privacy. 159 of the images were used for model training and 25 for validation. These two sets contained images with different rulers and different wounds. Most of the images contained rulers with scales up to 9 cm, but rulers that contain two-digit numbers are also present and the proposed method handles this.

Mean average precision (mAP) (2) is calculated based on the Average precision (AP) (1) and is used as an evaluation metrics for the digit detection task.

$$AP = \int_0^1 P(R)dR \tag{1}$$

where P is Precision, R is Recall.

$$mAP = \frac{1}{N}\sum_{i=1}^{N} AP_i \tag{2}$$

N is the number of classes

I used Google Collaboratory with GPU for YOLOv5 training on the custom dataset and Weights & Biases to track metrics. Initial experiment setup was: 2000 epochs, batch size 16 and image size 1280x1280 px. Training stopped after the 459s epoch with the best loss of 358 and the best mAP_0.5 0.84841 and the best mAP_0.5-0.95 0.473.

B. Detection of laying on the same line digits

The very heart of the method is the following. Taking the digit object detection model as input and using simple linear algebra equations and a few iterations, produce candidate points as outputs for the third step.

The algorithm consists of a few steps:

- select two candidate digits (D1 and D2) from the detected ones.
- using (3), get the equation of line L1 that goes through the center points (P1, P2) of D1 and D2, respectively

$$\frac{x - x_1}{x_2 - x_1} = \frac{y - y_1}{y_2 - y_1} \tag{3}$$

where x_1, y_1 – coordinates of P1 and x_2, , y_2 – coordinates of P2.

- from the rest of the digits, using a modification of ray casting algorithm, search for those whose bounding boxes are intersected by L1. The visual illustration is in Fig. 2:

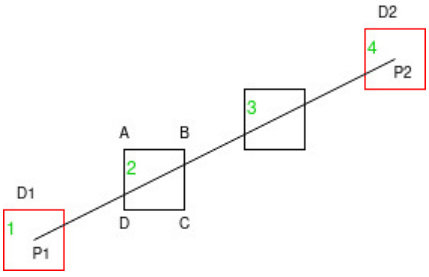


Fig. 2. Illustration of second method step main idea

where D1 and D2 are candidate digits (with labels 1 and 4), with P1 and P2 as digit centers, L1 is a line created using (2) that goes through points P1 and P2.

After that method verifies if L1 intersects with the other digits, 2 and 3, according to the example in Fig. 2. For example, digit 2 is represented by four points A, B, C, and D from which edges AB, BC, CD, and DA can be created using (3). By solving a system of two linear equations, we can find an intersection point and then check if it is in the range of points A and D. The same goes for the rest of the digits, that are being tested. We continue testing with different candidate digits D1 and D2 until we find that some portion of the digits that lay on the same line, which allows us to reject the falsely detected digits. Received digits are considered to be candidates for being ruler scale marks Fig. 3.

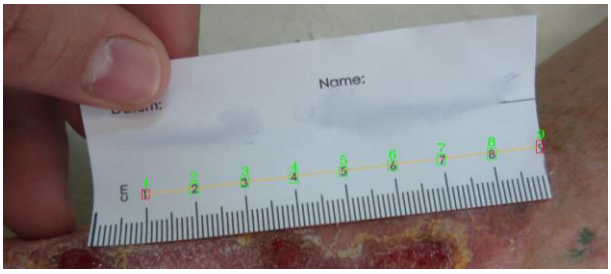


Fig. 3. Detected digits that lay on the same line

C. *Ruler scale detection*

The main assumption of the proposed method is that if there are many laying on the same line digits present on the image, – they represent a ruler. To discard false detections and increase the confidence of the first statement, two other checks are used: digits should be placed in consecutive order, and digits should form segments with equal lengths.

With digits that lay on one line, the calculation of distance in pixels can be done according to the following steps:

- Find consecutive digits only.
- Measure the pairwise distance between digit centers. For instance, if the method detects digits: [1, 2, 3, 4], it creates segments [(1-2), (2-3), (3-4)] and measures the length of each of them. To handle 2-digit numbers (10, 11, 12, etc.) that can also be present on the ruler, for each segment created in the previous step, we perform a special check, as in Fig. 4.

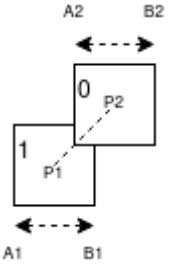


Fig. 4. Two-digit numbers detection

Where P1 and P2 correspond to the bounding box of detected digit 1 and bounding box of digit 0 centers, A1B1 – width of digit 1 and A2B2 – width of digit 0. Height check is also performed in the same ways as the width one.

If |A1B1| + |A2B2| > |P1P2| – these two digits are considered as one two-digit number. We check both digits width sum and digits height sum, and if at least one is longer than the distance between these digit centers – the decision that this is a two-digit number is taken. After that, two digits are processed as one.

- Use clustering to divide falsely detected segments from the true detected ones and perform digit detection only if there are at least 2 truly detected segments that have a similar length, taking into consideration tolerance “eps” selected experimentally.

I used a simplified version of the DBSCAN [8] clustering method; its density-based nature gives the ability to divide segments created by falsely detected digits from the right ones.

III. Results and Discussion

In Fig. 5 results of this method, automatic calculations of pixels per centimeter are present. The number above the blue line represents the number of pixels per 1 centimeter on the original image. This is a typical example of a good case when all digits are identified correctly, and there are not two-digit numbers.

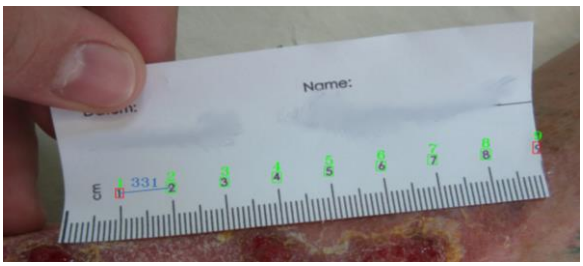


Fig. 5. Pixel per centimeter detection results

Results are worse when an image is of bad quality, but the method is still able to recognize the pixel per unit ration, but I cannot guarantee the accuracy, as even for manual annotation, these images are not suitable. An example is in Fig. 6.



Fig. 6. Pixel per centimeter, blured image detection results

If digits are blurred, the model still manages to recognize them correctly and detect pixels per centimeter. A more complex case is when two-digit numbers are present. Experiments show that this is a more challenging task for the digit detection model to detect digits in two-digit numbers. This can be slightly eliminated using the method from Fig. 3. Important note is that we should not consider only one segment length as suitable for decision-making results, more than one distance should be present. The result of worse case scale detection with the two-digit detection is present in Fig.7.



Fig. 7. Pixel per centimeter, two-digit image detection results

In Fig. 7, results were received just because of one-digit numbers being present. Our pre-trained YOLO model does not perform well for detecting digits that are close to each other, as well as rotated digits. Better digit detection model training should be introduced to improve the results.

I used Gimp photo editor and manually measured the pixels to centimeters ratio for all available 184 images. To ensure good measurement accuracy, I measured the distance between digit marks that are close to the center of the image and where the ruler is as close to the wound as possible and without any folds. The important point is that all images have a large resolution of 3104x1746 px, and the error of manual measurement between different digit marks on one image may differ in the range of [0-20] pixels or even more. Next, I computed pixels to centimeters using the proposed method and compared these results with the manual measurements, using the manual as a reference. The method was able to process 128 images from 184 available. The scale was not detected on the 56 images because less than 2 suitable segments was detected there. Error results (Fig. 8) show a

mean absolute error of 6.63 px and a median absolute error of 4 px.

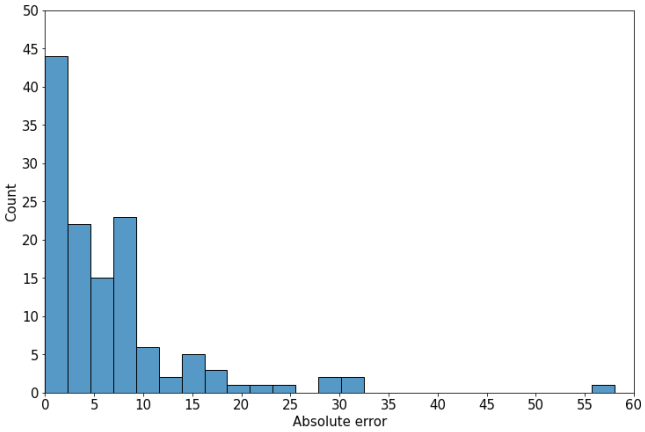


Fig. 8. Histogram of absolute errors between method and manual measurements

Maximal error – 59 px and minimal – 0 px. Fig. 8 also shows that the vast majority of errors is between 0 and 10 pixels, which is a suitable result compared with the manual measurement, and isolated cases have large errors of more than 30 px.

IV. Conclusions

The proposed method of ruler scale detection, unlike other methods that solve similar problem mostly using the transformation of image into the frequency domain, utilizes digit detection as a first step. Experiment results show that achieved accuracy of mean absolute error equal to 6.63 px (compared to manual measurement) can compete and overcome manual measurement results, thus, the method can be used to automate the process of ruler scale detection on the image in a variety of areas. During the development of the method, we found that the introduction of clustering of the created segment lengths, together with an iterative search of the segments that are used to calculate the distance in pixels, can significantly improve the accuracy of calculations and reject misidentified segments. Although this makes it impossible to determine the scale on some rulers, where the number of correctly detected digits is very small. It follows from the first step – digit detection. We used YOLOv5 trained and validated on the small dataset of 184 manually annotated images, which was enough to test and develop the method, however, we found that there are a few issues with it:

- the model poorly detects separate digits that lay close to each other (when digits represent two-digit numbers)
- it has problems detecting digits on the rotated images

While these problems can be easily fixed by improving model training, by introducing data augmentations, using different transformations, or just by using more images or increasing training data resolution, they also reveal one of the advantages of our method – the ability to use any digit detection method as a backend. This and also the fact that the method uses both one and two-digit numbers on the ruler, give users the ability to tune it for the domain they want and to receive the desired accuracy. In the future I am going to improve the proposed method with a better digit detection model and test it in conjunction with the wound segmentation method, too.

Acknowledgment

I express my respect and gratitude to all the brave ones defending Ukraine in the war with the russian federation.

References

[1] A. Bhalerao and G. Reynolds, “Ruler detection for autoscaling forensic images,” *Int. j. digit. crime forensics*, vol. 6, no. 1, pp. 9–27, 2014.

[2] D. A. Konovalov, J. A. Domingos, C. Bajema, R. D. White, and D. R. Jerry, “Ruler detection for automatic scaling of fish images,” in *Proceedings of the International Conference on Advances in Image Processing*, 2017.

[3] M. Herrmann, S. Zambanini, and M. Kampel, “Image based measurement of ancient coins,” *Tuwien.ac.at*. [Online]. Available: https://cvl.tuwien.ac.at/wp-content/uploads/2014/12/caa09.pdf.

[4] S. Ren, K. He, R. Girshick and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 39, no. 6, pp. 1137-1149, 1 June 2017, doi: 10.1109/TPAMI.2016.2577031.

[5] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779-788, doi: 10.1109/CVPR.2016.91..

[6] G. Jocher *et al.*, *ultralytics/yolov5: v3.1 - Bug Fixes and Performance Improvements*. Zenodo, 2020.

[7] N. Jaworski, I. Farmaha, U. Marikutsa, T. Farmaha and V. Savchyn, "Implementation features of wounds visual comparison subsystem," *2018 XIV-th International Conference on Perspective Technologies and Methods in MEMS Design (MEMSTECH)*, 2018, pp. 114-117, doi: 10.1109/MEMSTECH.2018.8365714.

[8] D. Deng, "DBSCAN Clustering Algorithm Based on Density," 2020 7th International Forum on Electrical Engineering and Automation (IFEEA), 2020, pp. 949-953, doi: 10.1109/IFEEA51475.2020.00199.